
Fort and Cards Project Report

Card Wars

Turn-Based Strategy Card Game

Developed by:

Achahoud Fatine
Fikry Mohamed Ali
Ikharrazen Abdelali
Lamjid Rabia

Supervised by:

M. Atlas Abdelghafour
M. Bekkari Aissam
M. Charef Ayoub

Contents

Abstract	iv
1 Introduction	1
1.1 Project Objectives	1
1.2 Scope	1
2 Project Overview	1
2.1 General Structure	1
2.2 Gameplay Experience	2
3 Game Concept & Rules	2
3.1 Turn Phases	2
3.2 Victory Conditions	2
4 Gameplay Walkthrough	2
4.1 Match Setup	2
4.2 Resource Management	3
4.3 Board Interaction	3
4.4 Turn Resolution	3
5 Card System & Archetypes	3
5.1 Character Cards	3
5.2 World Effect Cards	3
5.3 Spell Cards	3
6 Card Balance & Technical Sheets	3
6.1 Character Cards Profile	4
6.2 World Effect Cards Profile	5
6.3 Spell Cards Profile	5
7 Software Architecture	6
7.1 Data-Driven Design	6
7.2 Card Execution Pipeline	6
7.3 Hexagonal Coordinates	6

7.4	Utility-Based AI	7
8	UML Diagrams	7
8.1	Main Match Flow	7
8.2	Player Play Phase	8
8.3	Card Pipeline	9
8.4	Simplified Architecture	10
9	Diagram Interpretation	10
9.1	Main Match Flow	10
9.2	Player Play Phase	10
9.3	Card Pipeline	11
9.4	Class Diagram	11
10	Core Game Scripts	11
10.1	Key Scripts Directory and Roles	11
10.2	Detailed Script Descriptions	12
10.2.1	Core Loop: GameManager.cs	12
10.2.2	Board and Combat Grid: HexGrid.cs & UnitManager.cs	12
10.2.3	Computer Opponent: ComputerPlayer.cs & ComputerBrain.cs	12
10.2.4	Data Configurations: CardData.cs & ScriptableObjects	12
10.2.5	User Interface: HUDManager.cs & PlayerInputController.cs	13
11	AI Utility Model & Scoring Weights	13
11.1	Utility Scoring Formulations	13
11.2	Evaluation Criteria Weights Summary	14
11.3	AI Stopping Rule	15
12	Game Balancing Analysis	15
12.1	Global Parameter Configurations	15
12.2	Unit Stat-per-Gold Scaling Analysis	16
12.3	Rebalancing Key Assets	16
13	Technologies Used	16
13.1	Unity and C#	17
13.2	ScriptableObjects	17
13.3	UI, Audio, and Assets	17

14 Development Organization	17
14.1 Organization Highlights	17
15 Implementation Methodology	18
15.1 Key Development Steps	18
15.2 Technical Choices	18
16 Challenges	18
16.1 Mohamed Ali – Game Logic & Balancing	18
16.2 Abdelali – Hex Board & Combat Engineering	19
16.3 Rabia – Computer Opponent & Game UI	19
16.4 Fatine – Card System & Effects Architecture	20
17 Testing and Validation	20
17.1 Functional Testing	20
17.2 Game Balancing	20
17.3 UI Verification	21
18 Project Limitations	21
19 Future Enhancements	21
20 Conclusion	21
21 Educational Assessment	21
21.1 Acquired Skills	22
A Appendix A – Phase Summary	23
B Appendix B – Card Categories	23
C Appendix C – Deck Synthesis	23
D Appendix D – Key C# Source Code Listings	23
D.1 ActionScoringSystem.cs – Heuristic Scoring Formula	23
D.2 ComputerPlayer.cs – Asynchronous Turn Orchestrator	25

Abstract

Card Wars is a local turn-based strategy card game built in Unity by a four-student team. It combines hex-grid tactics, a card economy, combat units, instant spells, and a local AI opponent.

The project focuses on a playable and organized first Unity/C# experience. It emphasizes clean data-driven design, a card execution pipeline, and a modular architecture that keeps gameplay rules separate from the user interface.

1. Introduction

This report presents the Card Wars project, a turn-based strategy game centered on defending a Fort and the tactical use of cards. The project was designed as a first complete implementation in Unity and C#, with the goal of building a clean, structured system rather than a purely experimental prototype.

The game relies on three simple ideas: a hexagonal board, a turn-based resource economy, and cards that directly influence the course of the match. This combination provides clear, readable gameplay that is rich enough to offer interesting strategic choices.

1.1. Project Objectives

- Create a complete, playable game against a computer opponent.
- Organize the game rules in a clean, modular way.
- Decouple game data from logic to facilitate balancing.
- Establish a clean technical foundation for future development.

1.2. Scope

This report focuses on the current version of the game. It describes the concept, card types, turn structure, software architecture, AI, and team organization. The goal is not to delve into excessive detail, but to maintain a comprehensive and coherent overview.

2. Project Overview

Card Wars is a local strategy game in which each player protects a Fort located on their side of the board. Players progressively build their hand, deploy units, trigger effects, and aim to reduce the opponent's Fort health to zero.

The game was designed as a turn-based tactical combat experience with a resource management dimension. Unlike a traditional automatic draw system, players spend currency to purchase cards based on their chosen cost, introducing an important economic decision to each turn.

2.1. General Structure

A match follows a simple loop:

1. Economy Phase.
2. Buy / Discard Phase.
3. Play Phase.
4. End Phase & Victory Verification.

2.2. Gameplay Experience

The gameplay combines three types of actions:

- Deploying units on the board.
- Placing terrain effects or structures.
- Casting instant spells.

3. Game Concept & Rules

The board is composed of hexagons. Each player starts with a Fort placed on their half of the grid and a starting hand of four cards. The starting composition is balanced by design: two Character cards, one World Effect card, and one Spell card.

3.1. Turn Phases

Income Phase. The player receives a guaranteed income. This rule provides a steady pace to the match and prevents empty turns.

Buy / Discard Phase. The player can purchase a card of a specific cost or discard an unused card to recover a small amount of resources. Purchases are intentionally limited to one card per turn.

Play Phase. The player uses cards to deploy units, construct terrain features, or trigger immediate effects. Units can also move and attack.

End Phase. The engine checks the status of the Forts. If no Fort has been destroyed, the turn passes to the other player.

3.2. Victory Conditions

A match ends when a Fort reaches 0 HP. The opponent is then declared the winner, and a game-over screen allows players to restart.

4. Gameplay Walkthrough

A match of Card Wars follows a consistent logic from start to finish. This stability helps players quickly grasp the actions available during their turns.

4.1. Match Setup

At startup, each player is assigned a base, a Fort, starting currency, and an initial hand. This setup establishes the initial match state without overwhelming the player with excessive rules.

4.2. Resource Management

Gold is central to the game's pacing. A guaranteed income each turn encourages players to plan ahead rather than relying on random draws. This adds a layer of tactical economic strategy.

4.3. Board Interaction

The hexagonal board structures movement and control zones. Units are not merely deployed and forgotten; they must navigate the grid, protect assets, and attack at key moments to leverage their positioning.

4.4. Turn Resolution

Once all valid actions are completed, the engine checks the opponent's Fort health and passes the turn. This loop continues until a Fort is destroyed.

5. Card System & Archetypes ---

The core gameplay relies on three card archetypes. Each plays a distinct role and has different placement constraints.

5.1. Character Cards

Character cards are used to summon combat units onto the board. These units can attack enemy units, structures, and the Fort.

5.2. World Effect Cards

World Effect cards place structures, resource-generating buildings, or defensive barriers on the board.

5.3. Spell Cards

Spell cards apply immediate effects: healing, damage, buffs, debuffs, or general utility.

6. Card Balance & Technical Sheets ---

To achieve a fully functional and structured tactical experience, the game defines 21 distinct card types in its library. Each is configured with balanced values. This section lists the technical specifications of all active game cards.

6.1. Character Cards Profile

Character units are placed in the deployment zone (2 boundary columns on each side) and are governed by standard movement and combat attributes.

1. **Miner** – Economic Scout

- **Stats:** Cost: \$4 | HP: 34 | Attack: 8 | Range: 1 | Speed: 4.
- **Role:** Rapid exploration, grid positioning, and economical resource collection.

2. **UFO Cow** – Resource Saboteur

- **Stats:** Cost: \$5 | HP: 30 | Attack: 5 | Range: 1 | Speed: 3.
- **Role:** A beefy target focusing on moving, basic combat, and dismantling enemy structures.

3. **European King** – Hero Unit

- **Stats:** Cost: \$5 | HP: 40 | Attack: 5 | Range: 1 | Speed: 3.
- **Role:** Powerful hero unit with the specialized "colonization" ability to claim board tiles and capture enemy structures.

4. **Knight** – Heavy Melee Frontliner

- **Stats:** Cost: \$3 | HP: 58 | Attack: 16 | Range: 1 | Speed: 2.
- **Role:** High-health ground combatant for pushing through lanes.

5. **Archer** – Ranged Support

- **Stats:** Cost: \$3 | HP: 32 | Attack: 10 | Range: 3 | Speed: 2.
- **Role:** Essential counter to flying units, firing from a distance of 3 tiles.

6. **Spear Fighter** (Spearman) – Long-Reach Combatant

- **Stats:** Cost: \$3 | HP: 42 | Attack: 10 | Range: 2 | Speed: 2.
- **Role:** Versatile ground fighter capable of striking targets 2 tiles away.

7. **Bomber** – Siege Unit

- **Stats:** Cost: \$4 | HP: 38 | Attack: 14 | Range: 2 | Speed: 3.
- **Role:** Splash damage dealer, ideal for destroying walls and groups of defenders.

8. **Engineer** – Construction Support

- **Stats:** Cost: \$4 | HP: 44 | Attack: 0 | Range: 1 | Speed: 2.
- **Role:** Specializes in deploying and repairing defensive structures without direct offensive attacks.

9. **Priestess** – Ranged Healer/Fighter

- **Stats:** Cost: \$3 | HP: 34 | Attack: 10 | Range: 2 | Speed: 2.
- **Role:** Ranged unit capable of healing damaged allies or dealing combat damage within range.

10. **Dragon** – Flying Threat

- **Stats:** Cost: \$9 | HP: 50 | Attack: 15 | Range: 1 | Speed: 2.
- **Role:** Ultra-heavy late-game flyer. Immune to ground-based melee attacks.

6.2. World Effect Cards Profile

World Effects are placed on any empty tile on the player's side. They act as structures, defensive barriers, or income generators.

1. Wheat Field – Gold Generator

- **Stats:** Cost: \$3 | Structure HP: 20 | Effect: +1 gold income per turn.
- **Role:** Establishes an economic engine if defended.

2. Mines – Defensive Trap

- **Stats:** Cost: \$3 | Effect: Explodes dealing massive damage when crossed.
- **Role:** Secures pathways and chokepoints against invaders.

3. The Barrier (Wall) – Defensive Blockade

- **Stats:** Cost: \$3 | Structure HP: 20 | Effect: Blocks movement and ranged line-of-sight.
- **Role:** Chokes enemy movement lanes to protect the Fort.

4. Watch Tower – Automated Defense

- **Stats:** Cost: \$4 | Structure HP: 25 | Attack: 8 | Range: 2.
- **Role:** Deals damage automatically to adjacent hostiles.

5. Healing Station (Hospital) – Rejuvenation Center

- **Stats:** Cost: \$4 | Structure HP: 30 | Effect: Restores 6 HP to adjacent allies each turn.
- **Role:** Keeps defensive lines healthy without using Priestesses.

6. Anti-Air Tower – Sky Protector

- **Stats:** Cost: \$4 | Structure HP: 25 | Attack: 15 | Range: 2.
- **Role:** The primary defense system against the flying Dragon.

6.3. Spell Cards Profile

Spells do not spawn objects but apply instant combat modifications, resource theft, or crowd control.

1. Revival – Recovery

- **Stats:** Cost: \$5 | Target: Deceased Friendly Unit.
- **Role:** Chooses a deceased unit from the discard pile and places it directly back onto the board.

2. Freeze – Crowd Control

- **Stats:** Cost: \$3 | Target: Active Enemy Unit.
 - **Role:** Freezes an enemy unit, rendering it inactive for a full turn.
3. **Tax Collection** – Resource Theft
- **Stats:** Cost: \$3 | Target: Enemy Wheat Field.
 - **Role:** Targets an enemy Wheat Field structure and redirects its gold income to the player.
4. **Sabotage** – Structure Disable
- **Stats:** Cost: \$4 | Target: Active Enemy Structure.
 - **Role:** Disables a building's passives or attacks for one turn.
5. **Lightning Strike** – Direct Blast
- **Stats:** Cost: \$3 | Target: Single Board Hex.
 - **Role:** Deals 10 direct points of damage to the occupant.

7. Software Architecture

The project architecture is built on a clear separation of concerns between game data, rules, user interface, and effect execution. This enables changing gameplay behavior without rewriting the core codebase.

7.1. Data-Driven Design

Card statistics and various game settings are stored in *ScriptableObjects*. This approach avoids hardcoding values in scripts, facilitating easier balancing and long-term maintenance.

7.2. Card Execution Pipeline

When a card is played, the request passes through a precise pipeline:

`CardPlayRequest` → `Validator` → `Effect Factory` → `State Writer` → `CardPlayResult`

This decoupling minimizes errors and clarifies the responsibility of each component.

- **Validator:** Verifies if the action is valid/allowed.
- **Effect Factory:** Instantiates the corresponding effect.
- **State Writer:** Writes the resulting state changes to the game state.
- **CardPlayResult:** Returns the success or failure feedback of the action.

7.3. Hexagonal Coordinates

The board utilizes a *pointy-top* hexagonal grid with axial coordinates (q, r) . This choice simplifies neighborhood, range, and pathfinding calculations.

7.4. Utility-Based AI

The computer opponent does not follow a fixed sequence of actions. Instead, it evaluates multiple possible moves using a utility scoring system and selects the most optimal action based on the current match context.

8. UML Diagrams

This section presents the primary UML diagrams of the project. They have been preserved to document the original system design.

8.1. Main Match Flow

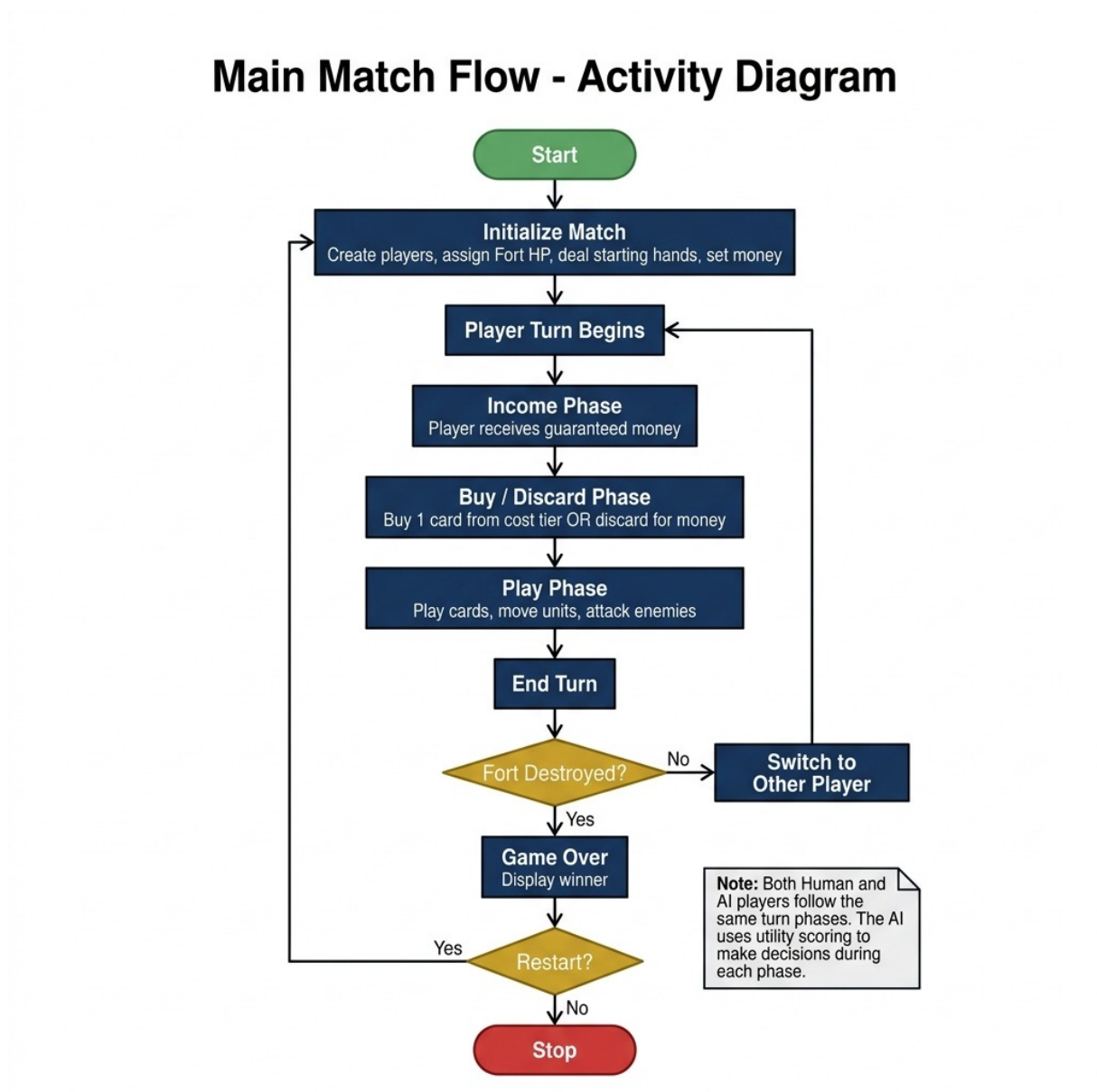


Figure 1: Main Match Flow – Activity Diagram

8.2. Player Play Phase

Player Play Phase - Activity Diagram

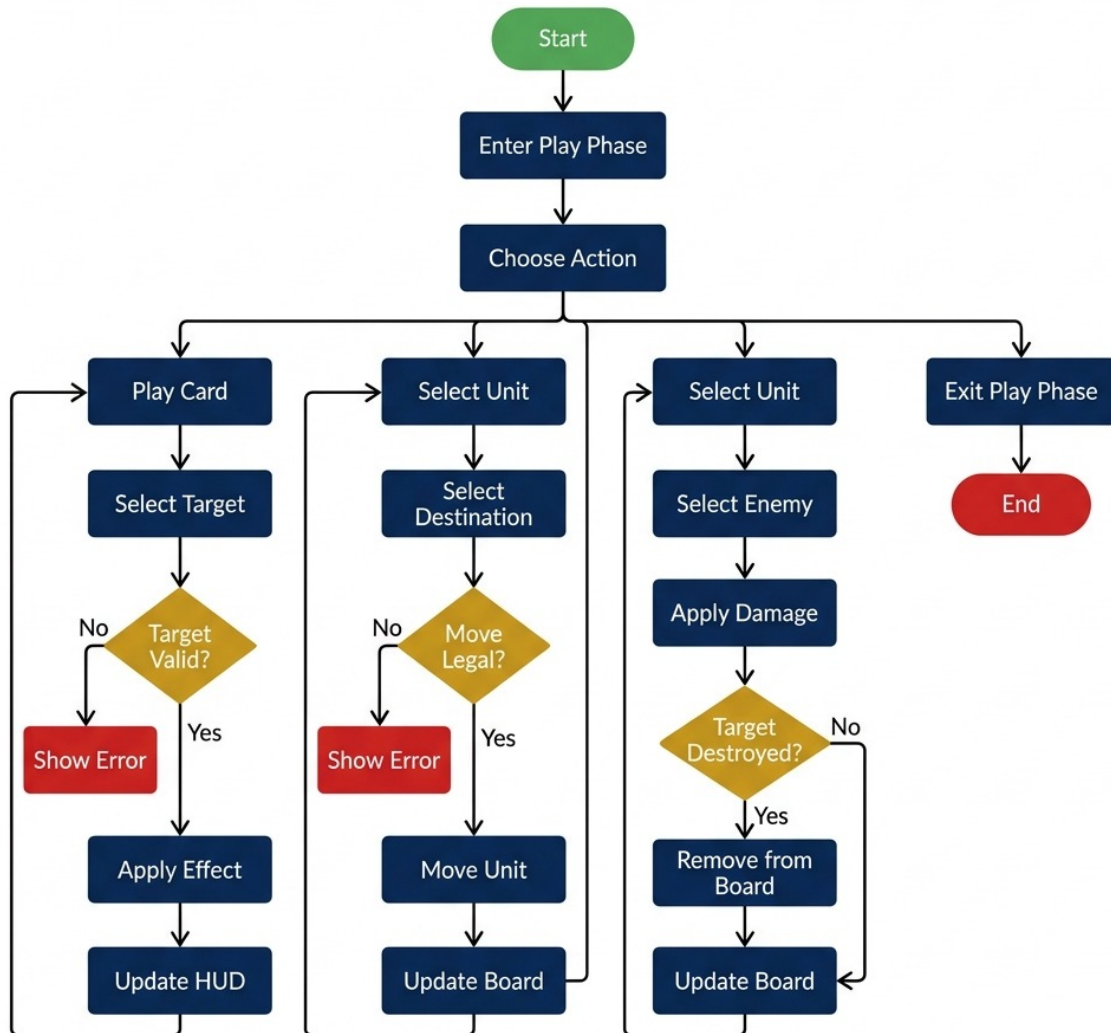


Figure 2: Player Play Phase – Activity Diagram

8.3. Card Pipeline

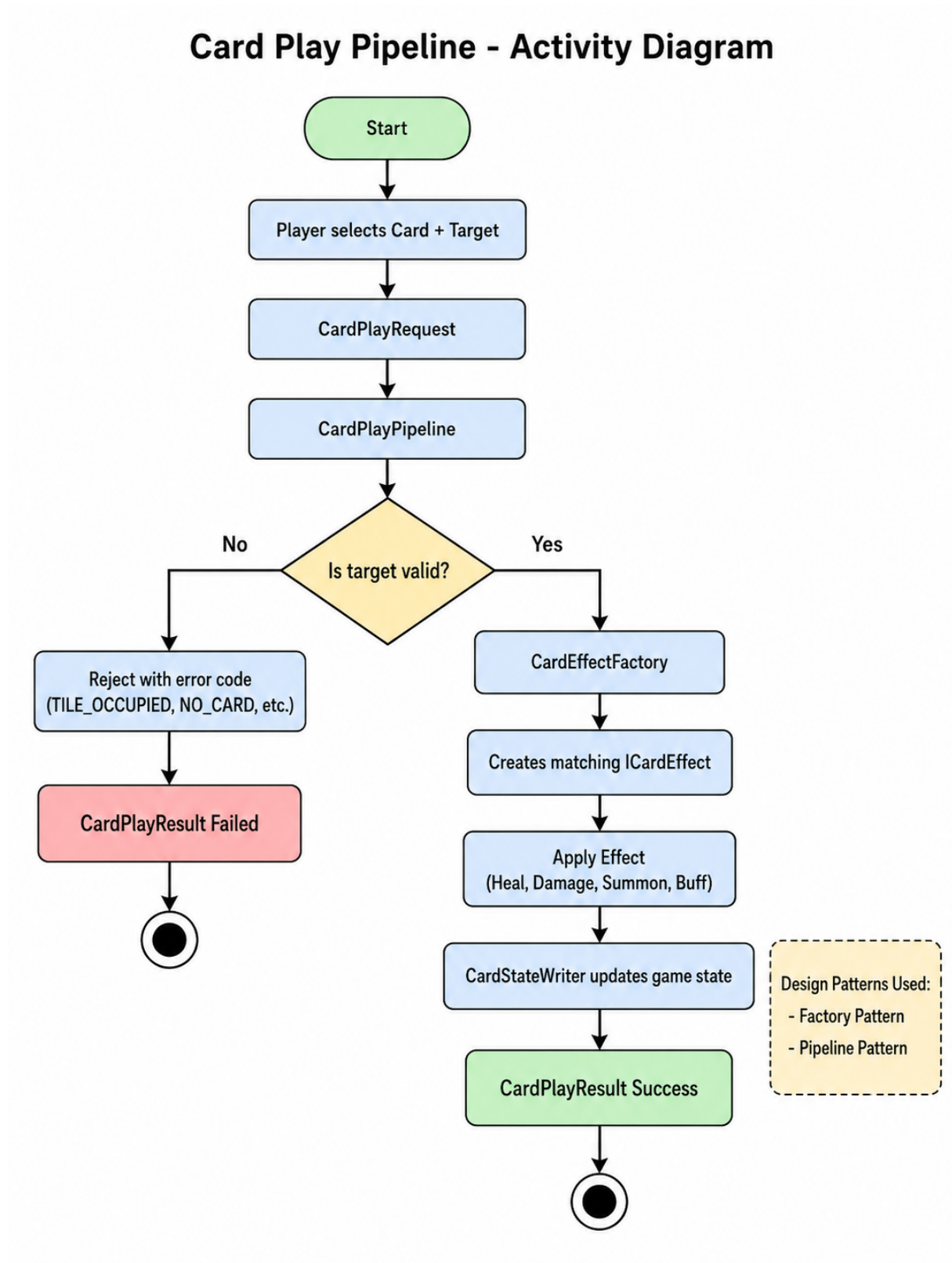


Figure 3: Card Play Pipeline – Activity Diagram

8.4. Simplified Architecture

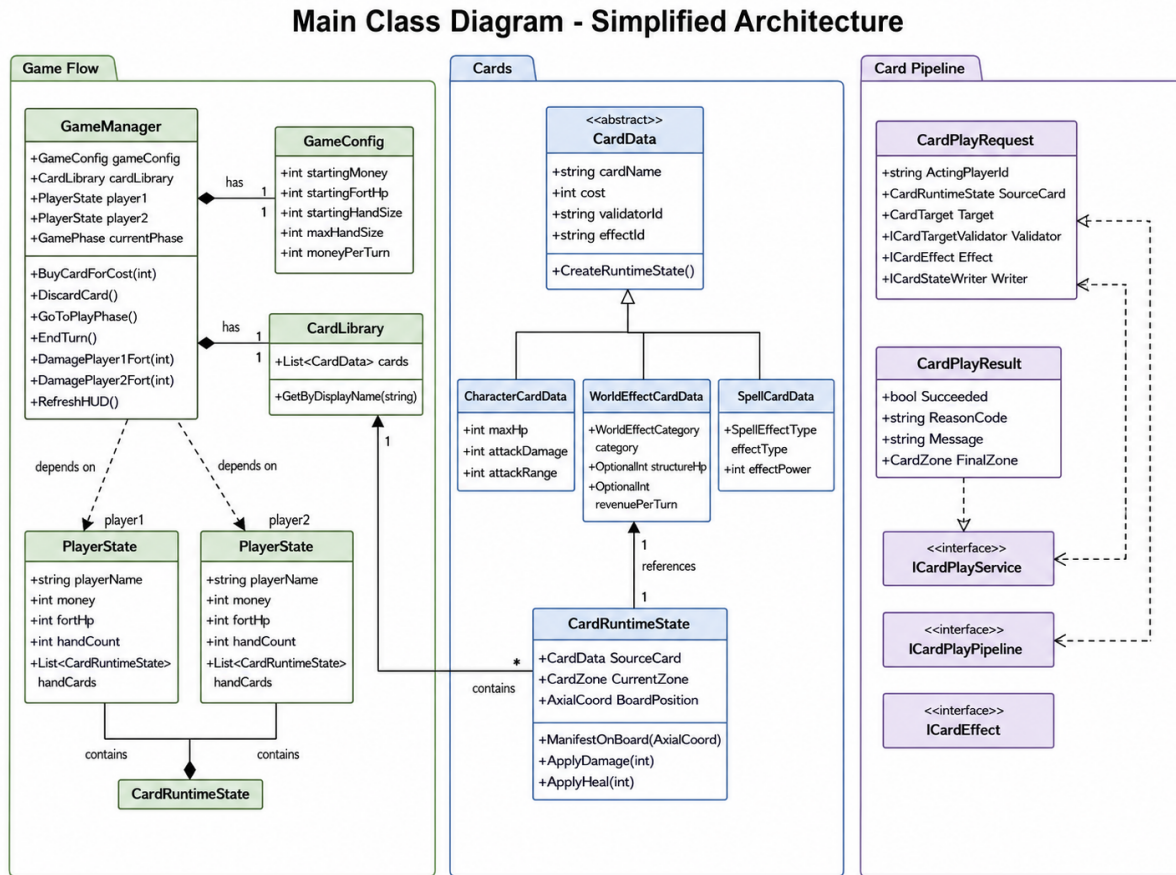


Figure 4: Main Class Diagram – Simplified Architecture

9. Diagram Interpretation

The UML diagrams visually summarize the project logic. They are included in this report because they offer a clear view of the game's behavior without requiring a line-by-line reading of the source code.

9.1. Main Match Flow

The main match flow diagram illustrates the overall game loop. It highlights the sequence of phases, the check for Fort destruction, and the transition to the next turn. This diagram provides the best representation of the game's global structure.

9.2. Player Play Phase

The play phase diagram details the actions available during a turn. It highlights the three primary choices: playing a card, moving a unit, or attacking. This view clarifies how actions flow within the user interface.

9.3. Card Pipeline

The card pipeline diagram displays the separation between validation, effect instantiation, and state writing. This is a key architectural feature, as it prevents gameplay rules from being tangled with display logic or user input handling.

9.4. Class Diagram

The class diagram summarizes the primary system objects: game manager, player states, cards, effect execution, and game services. It provides a straightforward view of how the codebase's main modules cooperate.

10. Core Game Scripts

The technical architecture of Card Wars is modularized into dedicated directories under the C# scripts folder, separating concerns across core flow, board management, card definitions, validation logic, computer behaviors, and user interfaces. This section details the primary scripts running the game's runtime architecture.

10.1. Key Scripts Directory and Roles

The table below summarizes the key scripts of the project and their primary architectural roles.

Directory	Script File	Primary Description / Role
Core/	GameManager.cs	Central game manager controlling phase transitions, player states, hand distribution, and win conditions.
board/	HexGrid.cs	Procedurally generates the board grid and converts screen inputs into hex cell indexes.
board/	HexTile.cs	Stores coordinates (q, r) , cell states (occupant, structure, owner), and tile color states.
board/	Unit.cs	Represents runtime character properties (HP, attack, range, movement) on the board.
board/	UnitManager.cs	Handles character movement paths, combat damage resolution, and dead unit cleanup.
Cards/	CardData.cs	Base class for ScriptableObject storing static templates for all game cards.
Computer/	ComputerPlayer.cs	Coordinates the AI player's turn loop using artificial action delays.
Computer/	ComputerBrain.cs	Implements the utility calculation logic to determine and execute high-value AI actions.

UI/	<code>HUDManager.cs</code>	Orchestrates visual displays (HP, gold, current player) and error banner announcements.
Validation/	<code>BoardPlacementRules.c</code>	Validates coordinate placements (e.g. restricting unit spawning to deployment zones).

10.2. Detailed Script Descriptions

10.2.1. Core Loop: `GameManager.cs`

The `GameManager` script coordinates the entire match lifecycle. It holds instances of the players' `PlayerState` scripts, the hexagonal grid, and the active configurations. It executes phase transitions inside the `GamePhase` state machine, distributing money during the `Income` phase, managing hand limits and purchases during the `Buy/Discard` phase, and checking for Fort destruction at the end of each turn.

10.2.2. Board and Combat Grid: `HexGrid.cs` & `UnitManager.cs`

The board is managed as a pointy-top hexagonal coordinate system:

- `HexGrid.cs` instantiates individual `HexTile` cells and arranges them axially. It exposes search helpers for neighboring tiles within specific ranges, which are vital for range highlighted displays and AI targeting.
- `UnitManager.cs` governs movement actions and combat interactions. It checks movement cost budgets against character capabilities, draws path highlights, and processes physical combat actions (resolving melee vs. ranged damage, updating health parameters, and calling cell cleanup routines).

10.2.3. Computer Opponent: `ComputerPlayer.cs` & `ComputerBrain.cs`

The AI player runs asynchronously to ensure smooth visual rendering:

- `ComputerPlayer.cs` serves as the AI controller, running a coroutine loop that performs actions sequentially with brief delays, letting players follow the machine's tactical decisions.
- `ComputerBrain.cs` uses the utility score formulas. It fetches a snapshot of the grid, calls generators to list all legal actions (unit movements, attacks, spell plays), calculates a numeric score for each choice based on combat threat ratios, and executes the best-scoring move.

10.2.4. Data Configurations: `CardData.cs` & `ScriptableObjects`

To decouple game assets from hardcoded scripts, all card configurations inherit from `CardData.cs`. Sub-classes like `CharacterCardData.cs`, `SpellCardData.cs`, and `WorldEffectCardData.cs` al-

low designers to tweak costs, damage parameters, and visual sprites directly in the Unity Inspector.

10.2.5. User Interface: HUDManager.cs & PlayerInputController.cs

The visual state of the game is handled by decoupled scripts:

- **HUDManager.cs** updates active canvas elements (health sliders, coins text, turn overlays).
- **PlayerInputController.cs** manages user raycasts, map hover highlights, and drag-and-drop operations for playing cards from the hand.

11. AI Utility Model & Scoring Weights

The computer opponent does not make choices based on fixed logic. Instead, its behavior is governed by a mathematical utility model implemented in **ComputerBrain.cs** and evaluated through **ActionScoringSystem.cs**. This scoring model calculates the overall utility of each possible action, sorting actions in descending order of value.

11.1. Utility Scoring Formulations

For each legal action a available to the computer in the current game state s , a total utility score $U(a, s)$ is computed. The total score is a combination of base utility, defensive priorities, combat trade values, positioning dynamics, tempo scaling, and card advantage, defined as:

$$U(a, s) = U_{\text{base}}(a) + U_{\text{defense}}(a, s) + U_{\text{combat}}(a) + U_{\text{position}}(a, s) + U_{\text{tempo}}(a, s) + U_{\text{card}}(a, s)$$

Each component is calculated based on the following specific heuristics:

1. **Base Utility (U_{base}):** Sets a baseline utility for playing card archetypes and special triggers.
 - Summoning a character unit: $U_{\text{base}} = +45$
 - Placing a defensive world structure: $U_{\text{base}} = +35$
 - Placing an offensive world structure: $U_{\text{base}} = +10$
 - Ending the turn: $U_{\text{base}} = -1000$ (ensuring it is only chosen when no positive actions remain)
2. **Defensive Priority (U_{defense}):** Triggered when an action is flagged as a defensive move (e.g., blocking an incoming lane or healing). It scales exponentially with the damage sustained by the AI's Fort:

$$U_{\text{defense}}(a, s) = \begin{cases} 40 + 30 \cdot (HP_{\text{max}} - HP_{\text{current}}) & \text{if } a \text{ is defensive} \\ 0 & \text{otherwise} \end{cases}$$

where $HP_{\text{max}} = 60$ is the starting Fort health, and HP_{current} is the AI's active Fort health.

3. **Combat Trade Utility** (U_{combat}): Evaluates the outcome of direct unit attacks or targeted damaging spells:

$$U_{\text{combat}}(a) = \begin{cases} +300 & \text{if action destroys enemy unit and actor survives} \\ +100 & \text{if action destroys enemy unit and actor dies} \\ -100 & \text{if actor dies without destroying the target} \\ 0 & \text{otherwise} \end{cases}$$

4. **Positioning & Pathfinding** (U_{position}): Evaluates movement and tactical placement on the board:

$$U_{\text{position}}(a, s) = T(a) + S(a) + M(a, s)$$

where:

- $T(a)$ is the raw tactical score computed by the pathfinding grid generator.
- $S(a)$ is a board synergy bonus (+75 if allied structures or buffs are nearby).
- $M(a, s)$ is the forward movement utility:

$$M(a, s) = \begin{cases} +15 & \text{if moving closer to enemy Fort and AI Fort } HP < 8 \\ +50 & \text{if moving closer to enemy Fort and AI Fort } HP \geq 8 \\ -50 & \text{if moving backward (unless defensive)} \end{cases}$$

5. **Tempo & Phase Pacing** (U_{tempo}): Modulates card values based on the progression of the match:

$$U_{\text{tempo}}(a, s) = \begin{cases} +60 & \text{if early-game card and turn } < 5 \\ -30 & \text{if early-game card and turn } \geq 5 \\ +80 & \text{if late-game card and turn } \geq 5 \\ -25 & \text{if late-game card and turn } < 5 \end{cases}$$

6. **Card Advantage** (U_{card}): Values hand replenishing and economy management:

$$U_{\text{card}}(a, s) = \begin{cases} 80 \cdot D & \text{if drawing } D \text{ cards and hand size } \leq 2 \\ 30 \cdot D & \text{if drawing } D \text{ cards and hand size } > 2 \\ +40 \cdot F & \text{if forcing opponent to discard } F \text{ cards} \\ +25 & \text{if playing unit card and hand size } \geq 4 \text{ (preventing card waste)} \end{cases}$$

11.2. Evaluation Criteria Weights Summary

The relative priorities of the AI decisions are summarized in the table below, representing the active weighting coefficients implemented in the AI scoring codebase.

Decision / Trade Action	Score Effect	Strategic Goal
Guaranteed Game Win (Destroy Fort)	+ 10000	Instant victory.
Perfect Unit Trade (Kill & Survive)	+ 300	Board dominance.
Even Unit Trade (Kill & Die)	+ 100	Combat parity.
Bad Unit Trade (Die & No Kill)	- 100	Prevent asset waste.

Defensive Move (Critical Fort HP)	+40 + 30/HP	Survival priority.
Board Synergy Placement	+75	Combined effects.
Push Unit Forward (Normal)	+50	Advance lane pressure.
Push Unit Forward (Fort Endangered)	+15	Slow advance.
Play Combat Unit (Normal)	+45	Summon unit.
Play Combat Unit (Hand Congested ≥ 4)	+70	Free hand slots.
Play Defensive Structure (Wall/Tower)	+35	Lane blockage.
Play Offensive Structure	+10	Field setup.
Play Early-Game Card (Turn < 5)	+60	Early tempo.
Play Early-Game Card (Turn ≥ 5)	-30	Late-game decay.
Play Late-Game Card (Turn ≥ 5)	+80	Late-game threats.
Play Late-Game Card (Turn < 5)	-25	Early bottleneck.
Draw Cards (Hand Size ≤ 2)	+80 per card	Desperation draw.
Draw Cards (Hand Size > 2)	+30 per card	Economy maintenance.
Force Opponent Discard	+40 per card	Resource denial.
Move Backward (Non-Defensive)	-50	Prevent retreat.

11.3. AI Stopping Rule

To prevent the computer player from executing meaningless or counter-productive actions, the `ComputerBrain.cs` script checks the maximum scoring action against a dynamic stopping rule.

If a guaranteed win is available, it is executed immediately. For standard actions, the brain compares the highest utility value $U_{\max}$ against a safety threshold θ :

$$\text{Action Choice} = \begin{cases} \text{Execute } a_{\text{best}} & \text{if } U_{\max} \geq \theta \\ \text{End Turn} & \text{if } U_{\max} < \theta \end{cases}$$

The default safety threshold is calibrated to $\theta = 5$. If all possible legal moves generate a utility score below 5 (or if the only legal action available is End Turn, which carries a score of -1000), the AI terminates its play phase and passes control back to the human player. This ensures the AI does not make random or wasteful moves.

12. Game Balancing Analysis

Balancing a card strategy game is necessary to ensure that players can explore distinct strategies (such as immediate rushing vs. economic buildup) without encountering dominant loops.

12.1. Global Parameter Configurations

The table below highlights the differences between the unplayable baseline configurations and the finalized, balanced values.

Rule Setting	Baseline	Balanced	Pacing Result
Fort Starting Health	45 HP	60 HP	Prevents early Knight rushes.

Starting Gold	\$4	\$5	Allows immediate cost choices.
Gold Revenue per Turn	\$3	\$2	Normalizes card purchases.

12.2. Unit Stat-per-Gold Scaling Analysis

The character stats follow a mathematical progression where stats are scaled in proportion to their gold cost.

- **Standard Combat Tier (\$3):** Knights (58 HP, 16 ATK, 1 range), Archers (32 HP, 10 ATK, 3 range), Priestesses (34 HP, 10 ATK, 2 range), and Spear Fighters (42 HP, 10 ATK, 2 range) are balanced against each other. The Knight dominates in raw statistics for melee combat, but the Spear Fighter's and Archer's extra range allow them to strike from behind cover.
- **Siege, Scout, and Support Tier (\$4):** Miners (34 HP, 8 ATK, 4 speed), Bombers (38 HP, 14 ATK, 2 range), and Engineers (44 HP, 0 ATK, 1 range) trade raw combat stats for high utility, map scouting, or area control.
- **High-Tier and Hero Units (\$5):** UFO Cows (30 HP, 5 ATK, 3 speed) and the European King (40 HP, 5 ATK, 3 speed) offer powerful disruption capabilities or colonization of board spaces.
- **Late-game Boss Tier (\$9):** The Dragon (50 HP, 15 ATK, 2 speed, flying) requires massive economic saving but dominates the late-game board.

12.3. Rebalancing Key Assets

Several assets were tuned to make them viable:

- **Lightning Strike Spell:** The baseline damage was 3 (for \$3), which was useless against units with 40+ HP. The damage was increased to 10, making it an effective tool to finish off weakened targets.
- **Healing Station:** Baseline healing was 2 HP. This was boosted to 6 HP/turn to make it a worthwhile defensive investment.
- **Defensive Structures:** The baseline HP of the Wall was 5. A single Knight (16 ATK) could destroy it instantly, rendering the card useless. The HP of The Barrier (Wall) was increased to 20 to enforce blocking strategy.

13. Technologies Used

The project was realized using standard game development tools, selected for their simplicity and suitability for an introductory Unity project.

13.1. Unity and C#

Unity provided the foundation for rendering, scene management, user interface, and game objects. The C# scripts implement the core logic: turns, cards, combat, resources, and AI. This engine/language combination ensured a direct and efficient development workflow.

13.2. ScriptableObjects

ScriptableObjects were used to store card statistics and general parameters. This approach separates content from code, making it easy to tune or add cards without altering the core codebase.

13.3. UI, Audio, and Assets

The project utilizes UI elements to display the player's hand, health points, currency, and turn state. Visual assets for cards and the board were integrated into the project with a unified theme to maintain visual clarity during gameplay.

14. Development Organization

The project was divided among four team members to minimize development conflicts and ensure clear ownership of each subsystem.

Member	Role	Primary Responsibilities
Mohamed Ali	Game Logic & Balancing Lead	Main match loop, turn management, currency, Fort HP, card hand, victory rules, balancing, and V1 documentation.
Abdelali	Hex Board & Combat Engineer	Hexagonal grid, tile system, deployment zones, unit movement, range, combat damage, cleanup of defeated units, and air/anti-air rules.
Rabia	Computer Opponent & Game UI	Full utility AI, action scoring, HUD, player hand, card selection, targeting, error feedback, and game-over screen.
Fatine	Card System & Effects Architect	Card classes, <i>ScriptableObject</i> schemas, card execution pipeline, effect factory, healing, damage, summon, buff, and economic effects.

14.1. Organization Highlights

This division allowed each developer to focus on a cohesive module. The resulting codebase is more readable and features fewer direct interdependencies.

15. Implementation Methodology

The project was built incrementally. The team first established the board grid and general turn flow, then introduced cards, active effects, combat mechanics, the user interface, and finally the utility AI.

15.1. Key Development Steps

- Hexagonal board prototyping.
- Implementing turns and resources.
- Integrating the card system.
- Developing the validation and effect pipeline.
- Adding the HUD and user feedback.
- Integrating the utility-based AI.

15.2. Technical Choices

The primary technical selections remained straightforward:

- Unity for the game engine.
- C# for core logic.
- *ScriptableObjects* for data storage.
- Modular architecture to maintain clean code separation.

16. Challenges

Developing a complex, turn-based tactical card game with an active AI opponent and data-driven assets in Unity presented several technical and design challenges. This section details the specific challenges faced by each team member based on their designated subsystems and codebases, along with the solutions implemented.

16.1. Mohamed Ali – Game Logic & Balancing

As the Game Logic & Balancing Lead, the primary difficulties centered on state synchronization, phase control, and gameplay pacing.

- **Game Loop Phase Stability:** Managing the transition between phases (**IncomePhase**, **BuyPhase**, **PlayPhase**, and **EndPhase**) in the main match controller. It was critical to prevent out-of-order execution, such as playing cards during the economic phase or purchasing cards when the turn had already transitioned.
- **Gameplay Balancing and Match Pacing:** The initial configuration had severe imbalances. For instance, the Fort starting health was set to 45 HP. A single heavy unit

like the Knight (16 ATK) could destroy the Fort in three turns, which made long-term economic strategies (such as deploying the Wheat Field building) useless. Similarly, the Lightning Strike spell was underpowered, dealing only 3 damage for a cost of 3 coins. To resolve these, the Fort HP was increased to 60, starting gold was adjusted to 5, and spell powers were calibrated (e.g., Lightning Strike damage increased to 10) to ensure meaningful choices.

- **Hand Management Constraints:** Implementing the "Full Hand" rule during the buy phase. When a player has a full hand of 7 cards and purchases a new one, the system had to block the immediate addition of the card, prompting a decision to either discard an existing card or cancel the purchase.

16.2. Abdelali – Hex Board & Combat Engineering

The Hex Board & Combat Engineer faced geometric, collision-related, and state-management challenges on the board.

- **Hexagonal Coordinate Calculations:** Converting the visual pointy-top grid into an axial coordinate system (q, r) to handle navigation, range calculations, and neighbors. Standard Cartesian math is not directly applicable, so pathfinding and range detection had to be rewritten using hex-vector algebra.
- **Flight and Anti-Air Combat Logic:** Implementing the flight properties of the *Dragon* unit. This required a custom targeting validator to ensure that ground-based melee units (like the Knight and Spearman) cannot select the Dragon as a target, whereas ranged units (Archers) and the specialized Anti-Air Tower can target it normally.
- **Incremental Movement System:** Allowing units to spend their movement points incrementally during the play phase (e.g., moving one tile, taking an action, and moving again) instead of forcing them to use all movement capacity in a single pathfinding call.
- **Structure Cleanup and Memory Management:** Ensuring that when a structure (World Effect) is destroyed, its entry in the hex tile is cleaned up. Failing to do so left "ghost" objects that visually disappeared but kept the tile occupied in code, blocking other units.

16.3. Rabia – Computer Opponent & Game UI

The Computer Opponent & Game UI developer dealt with AI decision-making algorithms and contextual user interfaces.

- **Utility-Based Decision Scoring:** Designing the scoring weights for the AI's `ComputerBrain`. Instead of a static behavior tree, the AI evaluates every legal action (summoning, moving, casting a spell, or attacking) and assigns it a utility score. The difficulty was preventing the AI from performing low-value actions (like attacking high-health units with weak ones) and writing a "stopping rule" so the AI passes its turn when no high-utility actions remain.
- **AI Spell Targeting Limitations:** The initial AI validator restricted spell casting to the enemy Fort. The AI could not target units on the board with utility spells (such as

Freeze or Revival). The validator had to be overhauled to allow targeting of specific allied or enemy unit states.

- **AI Economy and Discard Routines:** Since the AI originally had no discard routine, a full hand of unplayable cards would paralyze its turn loop, as it could not buy new cards. A system was implemented to force the AI to discard its lowest-utility card when its hand is full, freeing up slots and generating gold.
- **Contextual UI Feedback:** Creating visual target highlight overlays and clear error feedback messages (e.g., "Invalid Target", "Not Enough Gold") to guide the user in real-time.

16.4. Fatine – Card System & Effects Architecture

The Card System & Effects Architect focused on data modularity and pipeline execution integrity.

- **Modular Execution Pipeline:** Ensuring that playing a card follows a strict validation-before-execution flow (**CardPlayRequest** → **Validator** → **Effect Factory** → **State Writer**). The main challenge was maintaining decoupling so that adding a new card effect does not require modifying UI or grid controllers.
- **Data-Driven Modularity:** Structuring all card stats (cost, HP, damage, range) into *ScriptableObjects*. This required separating the static template data (**CardData**) from the runtime instance state (**CardRuntimeState**) to allow multiple instances of the same card to coexist on the board with independent health pools.
- **Factory Extensibility:** Designing a generic effect factory that dynamically instantiates and applies diverse card effects (summoning units, altering terrain, applying buffs/debuffs) based on configurable ID strings.

17. Testing and Validation

Testing was conducted iteratively during development by playing full matches to observe system behaviors in real-time.

17.1. Functional Testing

Primary verification focused on core rules: card costs, turn transitions, damage resolution, Fort destruction, and final match outcomes. Each module was validated before being integrated into the main system.

17.2. Game Balancing

Balancing adjustments focused on card costs, unit health, attack damage, and economic pacing. The goal was not perfect symmetry, but ensuring no card or strategy was disproportionately dominant or useless.

17.3. UI Verification

The UI was tested to ensure critical data (hand, gold, health, turn state, and errors) remained visible and readable, ensuring gameplay clarity.

18. Project Limitations

As an introductory complete project, Card Wars has some natural limitations:

- Content was kept limited to ensure project scope remained manageable.
- The AI could be more sophisticated and adaptive.
- Visual effects and animations could be further polished.
- Balancing requires further refinement through extended playtesting.

19. Future Enhancements

While the current version achieves its core goals, several future paths are naturally open:

- Expanding card variety.
- Polishing visual and audio feedback.
- Refining the AI logic.
- Adding more graphical feedback on the board.
- Strengthening general cost and damage balancing.

20. Conclusion

Card Wars represents a successful initial implementation of a turn-based strategy game in Unity. The project covers all gameplay essentials: hexagonal board, cards, turn phases, combat, economy, instant spells, and an AI opponent.

The primary value of the project lies not just in the playable game, but also in the clean separation of concerns, the use of external configurations (ScriptableObjects) for game content, and the design of a structured card execution pipeline.

As a first major project, the final deliverable provides a solid foundation for future extensions, while demonstrating a strong grasp of design and implementation fundamentals in Unity.

21. Educational Assessment

Beyond the game itself, the project served as an exercise in software engineering structure. It challenged the team to cleanly decouple data, rules, and UI, which is a crucial architectural habit for larger endeavors.

This effort also highlighted the benefits of subsystem-based development. When each module has a well-defined responsibility, testing, debugging, and expanding the project becomes significantly easier without breaking other modules.

21.1. Acquired Skills

- Game rules design.
- Object-oriented programming.
- Usage of Unity and C#.
- Data integration using *ScriptableObjects*.
- Collaborative teamwork across multiple technical domains.

A. Appendix A – Phase Summary

Phase	Summary
Income Phase	The player receives a fixed income to fund actions.
Buy / Discard Phase	The player purchases a card of a chosen cost or discards one for a small return.
Play Phase	The player deploys units, moves, attacks, or casts spells.
End Phase	The engine checks for Fort destruction and switches active players.

B. Appendix B – Card Categories

Category	Primary Role
Character	Spawns combat units.
World Effect	Deploys structures, utility buildings, and defenses.
Spell	Applies an immediate effect to the match state.

C. Appendix C – Deck Synthesis

This appendix provides a quick overview of the general deck contents. It allows readers to review the card design choices without going into the full details of every card.

Category	Summary
Units	Character cards deploy units that move forward, attack enemies, and defend the Fort.
Terrain	World Effect cards control the terrain, structures, and utility buildings.
Spells	Spell cards deliver immediate effects, useful for turning the tide of a turn.

D. Appendix D – Key C# Source Code Listings

This appendix presents the core C# implementation of the AI's action scoring system.

D.1. ActionScoringSystem.cs – Heuristic Scoring Formula

Below is the C# implementation of the heuristic scoring function, evaluating candidate game moves based on board context, safety budgets, and trade values.

```
// Core CalculateScore method from ActionScoringSystem.cs
private float CalculateScore(ComputerAction action, PlayerState myState, int currentTurn)
```

```

{
    float score = 0f;
    if (action == null) return float.MinValue;
    if (action.endsTurn || action.type == ActionType.EndTurn) return -1000f;
    if (action.willDestroyEnemyFort) return 10000f; // Guaranteed win

    // Save My Fort (Defensive priority)
    if (myState != null && action.isDefensiveMove)
    {
        int maxFortHp = 20;
        GameManager gm = Object.FindFirstObjectByType<GameManager>();
        if (gm != null && gm.gameConfig != null) maxFortHp = gm.gameConfig.startingFortHp;
        score += Mathf.Max(0, maxFortHp - myState.fortHp) * 30f + 40f;
    }

    // Favorable Unit Trades
    bool isUnitTradeAction = action.type == ActionType.AttackUnit || action.type == ActionType.AttackStructure
        || (action.type == ActionType.PlaySpellCard && (action.target.type == CardTargetType.EnemyUnit ||
            action.target.type == CardTargetType.EnemyStructure));
    if (isUnitTradeAction)
    {
        if (action.destroysEnemyUnit && action.survivesTrade) score += 300f;
        else if (action.destroysEnemyUnit && !action.survivesTrade) score += 100f;
        else if (!action.destroysEnemyUnit && !action.survivesTrade) score -= 100f;
    }

    // Board Control & Movement
    if (action.movesCloserToEnemyFort) score += myState != null && myState.fortHp < 8 ? 15f : 50f;
    if (action.hasSynergyOnBoard) score += 75f;
    score += action.tacticalScore;

    if (action.type == ActionType.PlayUnitCard)
    {
        score += 45f;
        if (myState != null && myState.handCount >= 4) score += 25f;
    }
    else if (action.type == ActionType.PlayWorldEffectCard) score += action.isDefensiveMove ? 35f : 10f;
    if (action.movesBackward && !action.isDefensiveMove) score -= 50f;

    // Economy & Tempo Scaling
    if (action.isEarlyGameCard) score += currentTurn < 5 ? 60f : -30f;
    if (action.isLateGameCard) score += currentTurn >= 5 ? 80f : -25f;

    // Card Advantage
    if (action.drawsCards > 0)
    {
        score += action.drawsCards * 30f;
        if (myState != null && myState.handCount <= 2) score += action.drawsCards * 50f;
    }
    if (action.forcesDiscard > 0) score += action.forcesDiscard * 40f;

    return score;
}

```

D.2. ComputerPlayer.cs – Asynchronous Turn Orchestrator

Below is the C# implementation of the AI player's turn loop, showing the coroutine structure used to execute actions with artificial delays.

```
// From ComputerPlayer.cs - Turn execution coroutine loop
private IEnumerator TurnLoopRoutine()
{
    yield return new WaitForSeconds(delayBetweenActions);
    int loopSecurity = 0;

    while (_isMyTurn && isActiveAndEnabled && loopSecurity < 20)
    {
        loopSecurity++;
        // Determine and execute the highest utility action
        bool hasActionToTake = _brain.DetermineNextAction(this);

        if (hasActionToTake)
        {
            yield return new WaitForSeconds(delayBetweenActions);
        }
        else
        {
            {
                break; // No more actions (e.g. out of money or cards)
            }
        }
    }
    EndTurn();
}
```